

L02 · CORALNPU 教学仓

L02 · 访存子系统 (LSU)

作业说明 —— 任务拆解、验收标准与调试方法

对应上游仓库: [google/coralnpu](https://github.com/google/coralnpu) (RTL、文档、测试程序都来自这里)

学习方式: 读规格 → 写 RTL → `./learn check` 对拍 → 修复

文档版本: 2026-09-12 · 生成命令: `./learn pdf`

目录

- [L02 作业说明](#)
 - [0. 交付物](#)
 - [1. 接口契约](#)
 - [2. 三个 Step](#)
 - [3. 验收标准](#)
 - [4. 调试方法](#)
 - [5. 常见坑](#)
 - [6. 进阶挑战 \(不计入验收\)](#)
 - [7. 学习记录 \(建议提交\)](#)

L02 作业说明

0. 交付物

只改一个文件: `course/lessons/L02_lsu/rtl/lsu.sv` (4 个 TODO)。

课程提供 (不要修改):

- `tests/tb_cpu.sv`: 测试平台, 支持 `+LATENCY=N` 模拟存储器延迟;
- `tests/core_wrapper.sv`: 核心部分 (译码/ALU/寄存器堆/提交逻辑);
- `tests/programs/06_unaligned.S`: 非对齐访问用例;
- L01 的 5 个测试程序 (会被一起跑)。

验收:

```
./learn check L02                # 期望 12/12 (6 个程序 × 2 种延迟)
./learn check L02 --only 03      # 只跑某个用例
./learn check L02 --ref          # 看参考实现
```

1. 接口契约

```
module lsu (
    input logic          clk, rst,
```

```

// 核心 → LSU
input logic      req_valid,      // 有访存指令要执行 (LSU 空闲时给一个脉冲)
input logic      req_write,      // 1=store, 0=load
input logic [31:0] req_addr,      // 字节地址 = rs1 + imm
input logic [2:0] req_funct3,     // 000=lb 001=lh 010=lw 100=lbu 101=lh
input logic [31:0] req_wdata,     // store 的原始数据 (未对齐、未扩展)
// LSU → 存储器
output logic      dmem_valid,
output logic      dmem_we,
output logic [31:0] dmem_addr,
output logic [3:0] dmem_wmask,
output logic [31:0] dmem_wdata,
input logic [31:0] dmem_rdata,
input logic      dmem_ready,
// LSU → 核心
output logic      busy,           // 1 = 没做完, 核心冻结
output logic      done,           // 1 = 本周周期完成 (载入结果在 rdata 上)
output logic [31:0] rdata
);

```

约定:

1. 请求只在 busy=0 时到来; 收到后要锁存 (req_* 不会一直保持);
2. dmem_valid 只能在 REQ 状态拉高; addr/wmask/wdata 在等到 ready 之前保持不变;
3. 载入完成时 rdata 必须是已重排并已做符号/零扩展的最终值;
4. done 只拉高一拍 (FINISH 状态), 然后回 IDLE。

2. 三个 Step

Step 1: 单笔对齐事务 (让 01~05 通过)

先不管延迟和非对齐, 把”一次对齐访问”做出来:

1. IDLE: 收到 req_valid 时锁存 req_write/funct3/addr/wdata, 跳到 REQ;
2. REQ: 给出

```

dmem_valid = 1'b1;
dmem_we    = is_write;

```

```

dmem_addr = {addr[31:2], 2'b00};           // 对齐到字
dmem_wmask = base_mask(func3) << off;      // 字节道
dmem_wdata = wdata << (8 * off);           // 数据对齐

```

等到 `dmem_ready` 后：写操作直接进 `FINISH`；读操作先存下 `word0 = dmem_rdata`；

3. `FINISH`: `done=1`, 下一拍回 `IDLE`;

4. 读结果: `shifted = {32'b0, word0} >> (8*off)`, 再按 `func3` 扩展。

自检: `./learn check L02 --only 03` (`03_mem` 是纯对齐访存)。

Step 2：握手等待 (`latency=2` 也要过)

如果 Step 1 是在“给地址的同一拍就采样数据”，那么在 `latency=2` 时会拿到错的字节。正确做法：只在 `dmem_ready=1` 的那一拍采样，并在等待期间保持输出不变。

自检: `./learn check L02 --only 03`, 两种延迟都要通过。

Step 3：非对齐拆分 (让 `06_unaligned` 通过)

1. 收到请求时算出 `off`、`size`、`need_two = (off + size) > 4`;
2. 第一笔完成后，如果 `need_two`，把 `step` 置 1、切换到第二笔参数 (`addr1/mask1/data1`)；
3. 读操作把第一笔结果存 `word0`、第二笔存 `word1`;
4. 最终 `combined = {word1, word0}`，再统一做右移与扩展。

自检: `./learn check L02 --only 06`。

3. 验收标准

`./learn check L02`

期望:

```

[通过] 01_arith.S      latency=0 / latency=2
[通过] 02_branch.S     latency=0 / latency=2
[通过] 03_mem.S        latency=0 / latency=2
[通过] 04_call.S       latency=0 / latency=2
[通过] 05_c_fib.c      latency=0 / latency=2
[通过] 06_unaligned.S  latency=0 / latency=2

```

12/12 次运行通过

4. 调试方法

第一招：看检查器报的是哪一类错。

检查器的说法	通常意味着
trace 行数不同	重复提交（done 拉了多拍）或漏提交（忘了拉 done）
某条 load 的 wdata 是 0	没等 ready 就采样，或根本没发起读事务
trace 一致，但 DTCM 有字不同	写掩码/数据对齐错，或非对齐只写了第一笔
超时	busy 没有落下，或状态机卡在 REQ

第二招：把事务打出来。在 REQ 状态等待时加一行，观察每次事务：

```
always_ff @(posedge clk) begin
    if (!rst && dmem_valid && dmem_ready)
        $display("[lsu] %s addr=%08x mask=%b data=%08x step=%0d",
                  dmem_we ? "WR" : "RD", dmem_addr, dmem_wmask, dmem_wdata, step);
end
```

06_unaligned.S 里 sw 0xAABBCCDD, 3(s5) 应该看到两行：mask=1000 和 mask=0111。

第三招：单独看某一条指令。用 --only 06 缩小范围；再用黄金模型打印期望值：

```
./learn check L02 --only 06 --no-color
```

5. 常见坑

现象	原因	修法
写进去的是 0	数据没左移 8*off	<code>dmem_wdata = wdata << (8*off)</code>
相邻字节被改坏	掩码没按 off 平移	<code>mask = base_mask << off</code>
latency=0 过、latency=2 挂	没等 ready 就采样/收工	只在 dmem_ready 那一拍采样，并保持输出不变
非对齐写只改了一半	忘了第二笔事务	

现象	原因	修法
第二笔掩码算错	移位方向反了	<code>need_two = (off + size) > 4, 第二笔地址 +4</code> <code>mask1 = base_mask >> (4 - off)</code>
非对齐读结果错位	忘了拼 64 位再右移	<code>combined = {word1, word0}, shifted = combined >> (8*off)</code>
同一指令提交两次	done 持续拉高	只在 FINISH 拉高，且只停留一拍
超时	busy 一直是 1	FINISH → IDLE 的转移漏了
01 就挂了	连对齐访问都没做	先按 Step 1 做单笔事务

6. 进阶挑战（不计入验收）

- 挑战 1（槽表）：照着 [doc/microarch/lsu.md](#) 的表格，把一次操作拆成“每个字节一个条目 + active 位”，逐条完成——这是上游的实现方式；
- 挑战 2（多 outstanding）：允许两条访存指令同时在途（提示：需要队列和 id）；
- 挑战 3（错误上报）：地址越界时把错误上报给核心（L03 的异常机制会用到）；
- 挑战 4（随机延迟）：改测试平台，让 ready 随机延迟 0~5 拍，验证仍然正确。

7. 学习记录（建议提交）

在 `course/work/L02/记录.md` 里写下：

1. 你在 Step 2（等待 ready）踩到的坑；
2. `06_unaligned.S` 里哪一条最让你意外，为什么；
3. 对照上游的 slot 表，说出你的实现与它的两点差异。